Dossier d'architecture — Portefeuille numérique UEMOA

Oct 28, 2026 · @SaifCores

Ce dossier décrit l'architecture qui réalise le CDC de l'étude de cas : cinq services métier, trois simulateurs de rails, une base PostgreSQL par service et un broker de messages, le tout lancé en local par Docker Compose.

1. Vue d'ensemble

L'architecture repose sur une règle : **seul le service ledger écrit l'état monétaire**, en une transaction de base de données qui insère les écritures et met à jour les soldes ensemble. Les autres services demandent des postings, réagissent à des événements et ne gardent jamais de solde.

Principes

1. **Ledger isolé.** Service et base dédiés ; aucune autre base ne contient de montant faisant foi.
2. **Écritures immuables.** Une écriture n'est jamais modifiée ; seul le statut de la transaction évolue (pending → posted ou archived), et une correction est une transaction inverse.
3. **Idempotence de bout en bout.** Clé d'idempotence à la passerelle, `external_id` déterministe au ledger, inbox dédoublonnée pour les callbacks.
4. **Au moins une fois + consommateurs idempotents.** Outbox transactionnelle vers le broker ; aucun événement perdu, doublons absorbés.
5. **Sagas orchestrées.** Chaque mouvement externe (top-up, cash-out, PI-SPI sortant) est une machine à états explicite dans le service paiements, jamais une chaîne implicite d'événements.
6. **Pannes comme fonctionnalité.** Les simulateurs exposent une API d'injection de pannes ; chaque panne a un test.
7. **Montants entiers.** `BIGINT` en XOF, exposant 0, partout — base, API, événements.

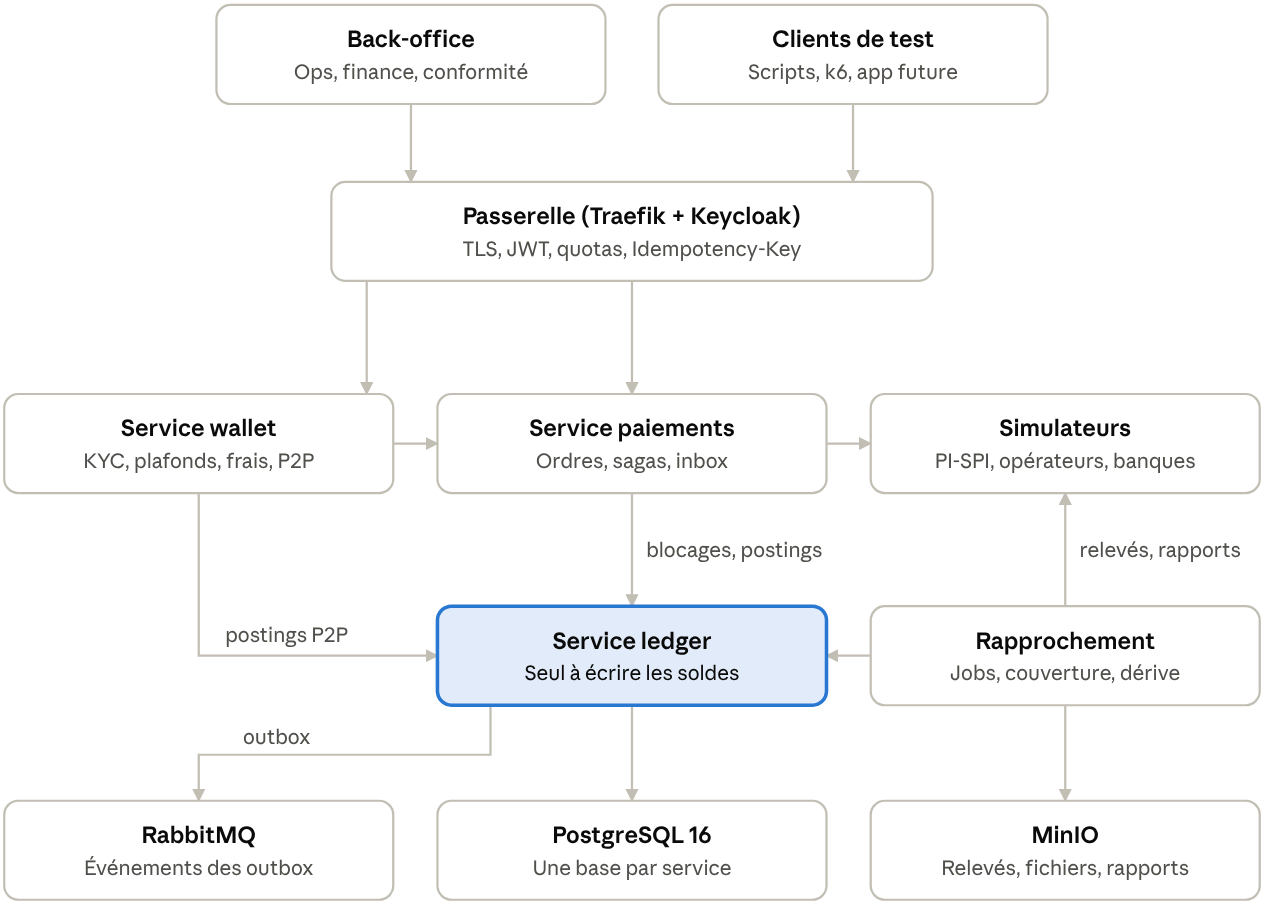
Décisions clés (détail en section 9)

Sujet	Choix
Découpage	Services : ledger, wallet (avec conformité en module), paiements, rapprochement, passerelle
Langage	Java 25 (LTS) + Spring Boot 4.x, monorepo Gradle multi-modules, DDD hexagonal (section 11)
Base	PostgreSQL 16, une base par service
Verrouillage	Verrous de ligne `FOR UPDATE` triés pour les comptes à conditions ; soldes asynchrones pour les comptes chauds
Messagerie	RabbitMQ (exchanges topic), alimenté par une outbox
Communication synchrone	HTTP/JSON interne (gRPC en option)
Stockage de fichiers	MinIO (relevés, fichiers de règlement, rapports)
Observabilité	OpenTelemetry → Prometheus, Grafana, Loki, Tempo

2. Composants

La passerelle ne route que vers les services wallet et paiements ; le ledger n'est joignable que depuis l'intérieur, par les trois services qui ont le droit de lui demander un posting.

Seul le service ledger écrit les soldes, les autres services orchestrent



composants · 5 services, 3 familles de simulateurs, 3 briques d'infrastructure

Le service wallet poste lui-même les P2P ; tout mouvement externe passe par le service paiements, qui pose un blocage au ledger avant d'appeler un simulateur. Le rapprochement lit le ledger et les simulateurs, et n'écrit jamais de solde. L'observabilité (OpenTelemetry, Prometheus, Grafana) couvre tous les conteneurs et n'est pas dessinée.

Composant	Responsabilités	Données possédées	Ne fait jamais
Passerelle	TLS, authentification JWT, quotas, présence de l'`Idempotency-Key`	—	Logique métier

Composant	Responsabilités	Données possédées	Ne fait jamais
Service wallet	Inscription, KYC simulé, niveaux et plafonds, grilles de frais, P2P, paiements marchands, règles AML, gels	Utilisateurs, wallets, niveaux, configuration datée, alertes	Calculer un solde lui-même
Service paiements	Ordres de paiement, sagas top-up / cash-out / PI-SPI, inbox des callbacks, enquêtes de statut, adaptateurs de rails	Ordres, états de saga, inbox	Poster sans `external_id` déterministe
Service ledger	Comptes, transactions, écritures, compteurs de solde, catégories, conditions de solde	Tout l'état monétaire	Modifier une écriture
Rapprochement	Jobs banques / PI-SPI / opérateurs, couverture, scan de dérive, agrégation des comptes asynchrones	Correspondances, écarts, rapports	Écrire un solde de wallet
Simulateurs	Imiter PI-SPI, deux opérateurs, deux banques, avec pannes injectables	Leur propre état (float, relevés)	Partager une base avec la plateforme

3. Modèle de données du ledger

Les écritures sont immuables ; les soldes vivent dans des compteurs par compte (quatre seaux) mis à jour dans la même transaction, et toujours recalculables depuis les écritures.

Table	Rôle	Points clés
`ledgers`	Un ledger par entité et environnement	`live` / `test`
`ledger_accounts`	Comptes	`normal_balance` D/C, devise figée, `balance_mode` sync/async
`account_balances`	Compteurs de solde	`posted_dr`, `posted_cr`, `pending_dr`, `pending_cr`, `lock_version`
`ledger_transactions`	Événements métier	`external_id` unique par ledger, statut, `request_hash`, `reverses_id`
`ledger_entries`	Lignes d'écriture	Immuables, partitionnées par mois
`balance_deltas`	Deltas des comptes asynchrones	Agrégés par un worker dans `account_balances`
`account_categories`, `category_members`	Agrégats	Monnaie en circulation, couverture…
`outbox`	Événements à publier	Écrite dans la même transaction
`audit_log`	Actions d'admin	Chaînée par hachage

Calcul des soldes depuis les compteurs (wallet normal créditeur ; inverser D et C pour un compte normal débiteur)

$$\text{posted} = \text{posted_cr} - \text{posted_dr}$$

$$\text{pending} = (\text{posted_cr} + \text{pending_cr}) - (\text{posted_dr} + \text{pending_dr})$$

$$\text{available} = \text{posted_cr} - (\text{posted_dr} + \text{pending_dr})$$

Quand une transaction passe de pending à posted, ses montants quittent les seaux `pending_*` pour les seaux `posted_*` ; archivée, ils quittent simplement `pending_*`.

DDL (extrait)

```
CREATE TYPE normal_balance AS ENUM ('debit', 'credit');
CREATE TYPE txn_status     AS ENUM ('pending', 'posted', 'archived');
CREATE TYPE direction      AS ENUM ('debit', 'credit');

CREATE TABLE ledger_accounts (
  id                uuid PRIMARY KEY,
  ledger_id         uuid NOT NULL REFERENCES ledgers(id),
  name              text NOT NULL,
  normal_balance    normal_balance NOT NULL,
  currency          char(3) NOT NULL DEFAULT 'XOF',
  currency_exponent smallint NOT NULL DEFAULT 0,
  balance_mode      text NOT NULL DEFAULT 'sync' CHECK (balance_mode IN
('sync','async')),
  status            text NOT NULL DEFAULT 'active',
  metadata          jsonb NOT NULL DEFAULT '{}',
  created_at        timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE account_balances (
  account_id   uuid PRIMARY KEY REFERENCES ledger_accounts(id),
  posted_dr    bigint NOT NULL DEFAULT 0 CHECK (posted_dr  >= 0),
  posted_cr    bigint NOT NULL DEFAULT 0 CHECK (posted_cr  >= 0),
  pending_dr   bigint NOT NULL DEFAULT 0 CHECK (pending_dr >= 0),
  pending_cr   bigint NOT NULL DEFAULT 0 CHECK (pending_cr >= 0),
  lock_version bigint NOT NULL DEFAULT 0,
  updated_at   timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE ledger_transactions (
  id           uuid PRIMARY KEY,
  ledger_id    uuid NOT NULL REFERENCES ledgers(id),
  external_id  text NOT NULL,
  request_hash bytea NOT NULL,
  status       txn_status NOT NULL,
  description  text,
  effective_at timestamptz NOT NULL,
  posted_at    timestamptz,
  reverses_id  uuid REFERENCES ledger_transactions(id),
```

```
  metadata     jsonb NOT NULL DEFAULT '{}',
  created_at   timestamptz NOT NULL DEFAULT now(),
  UNIQUE (ledger_id, external_id)
);

CREATE TABLE ledger_entries (
  id             bigint GENERATED ALWAYS AS IDENTITY,
  transaction_id uuid NOT NULL,
  account_id     uuid NOT NULL,
  direction      direction NOT NULL,
  amount         bigint NOT NULL CHECK (amount > 0),
  account_version bigint NOT NULL,   -- lock_version du compte après cette
écriture
  created_at     timestamptz NOT NULL DEFAULT now(),
  PRIMARY KEY (id, created_at)
) PARTITION BY RANGE (created_at);

CREATE INDEX ON ledger_entries (account_id, created_at);
CREATE INDEX ON ledger_entries (transaction_id);
CREATE INDEX ON ledger_transactions USING gin (metadata);
```

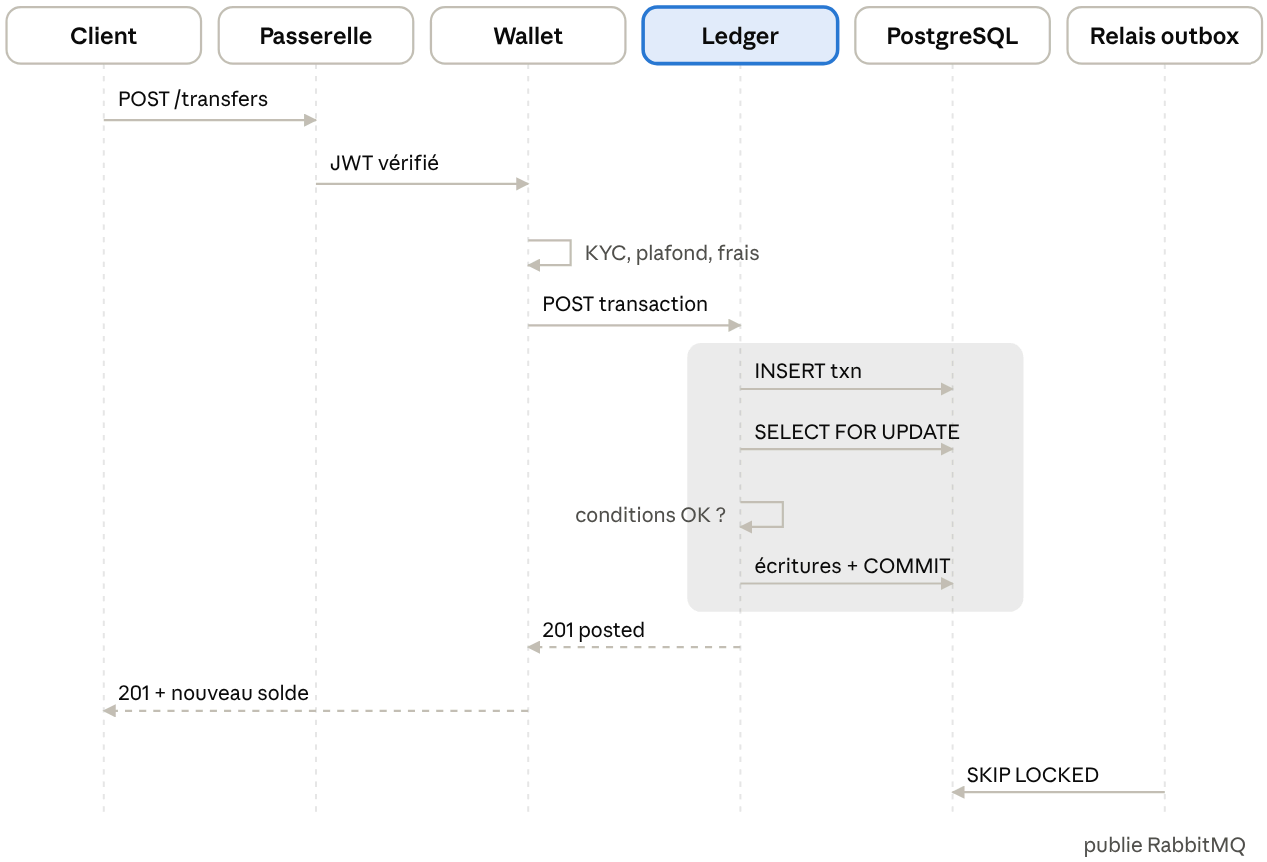
Garde-fous en base (en plus des contrôles applicatifs)

- Le rôle applicatif n'a ni `UPDATE` ni `DELETE` sur `ledger_entries` ; un trigger refuse aussi toute modification de transaction autre que `pending → posted|archived`.
- Un trigger de contrainte `DEFERRABLE INITIALLY DEFERRED` vérifie au commit que Σ débits = Σ crédits pour chaque transaction insérée.
- Les `CHECK (>= 0)` sur les compteurs interdisent un seau négatif, même en cas de bug applicatif.
- Requête de vérification nocturne : recalculer les quatre seaux depuis `ledger_entries` ⋈ `ledger_transactions` et comparer à `account_balances` (exercice 9 du CDC).

4. Chemin d'écriture

Un posting est une seule transaction PostgreSQL : insertion idempotente, verrous triés, contrôle des conditions, écritures, compteurs et outbox, puis commit ; en cas d'échec rien n'est écrit.

Une seule transaction PostgreSQL écrit écritures, soldes et outbox



séquence d'un P2P · zone grisée = une transaction PostgreSQL

Algorithme de posting (service ledger)

1. **Valider la requête hors transaction :** au moins deux écritures, montants > 0, Σ débits = Σ crédits, même devise, comptes actifs, aucune condition sur un compte `async`. Calculer `request_hash`.
2. `BEGIN` (READ COMMITTED).
3. `INSERT INTO ledger_transactions … ON CONFLICT (ledger_id, external_id) DO NOTHING RETURNING id`. Rien de retourné : relire la transaction ; même `request_hash` → `COMMIT` et renvoyer l'existante (200) ; sinon 409.
4. `SELECT … FROM account_balances WHERE account_id = ANY($sync_ids) ORDER BY account_id FOR UPDATE` : l'ordre fixe évite les interblocages entre deux P2P croisés.
5. Appliquer les écritures aux compteurs en mémoire (seaux `pending_*` si statut pending, `posted_*` sinon), puis évaluer chaque condition (`available >= 0`, `pending <= plafond`). Une condition fausse → `ROLLBACK`, 422 avec le compte et la condition en cause.

6. `INSERT` des écritures (avec `account_version`), `UPDATE account_balances SET …, lock_version = lock_version + 1` pour les comptes `sync`, `INSERT` dans `balance_deltas` pour les comptes `async`.
7. `INSERT` dans `outbox` de `ledger.transaction.posted` (ou `.pending`).
8. `COMMIT` : le trigger différé vérifie l'équilibre ; la réponse renvoie la transaction et les nouveaux soldes des comptes `sync`.

Transitions d'une transaction pending

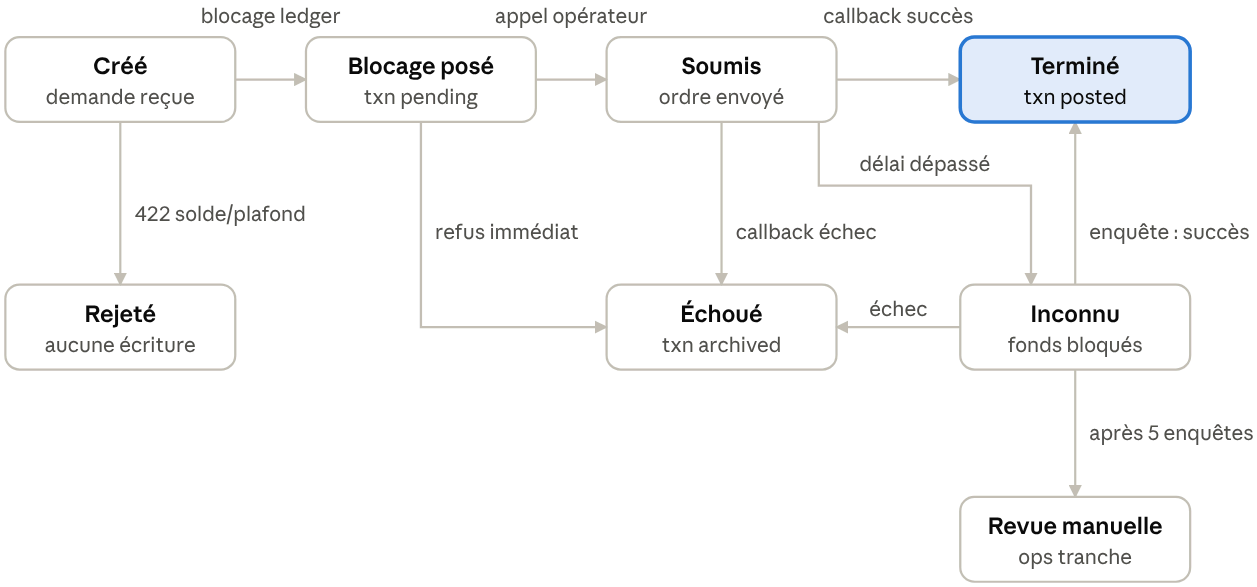
Transition	Effet sur les compteurs	Contrôles
pending → posted	Montants déplacés de `pending_*` vers `posted_*`	Verrous triés ; aucune condition réévaluée sur les débits déjà bloqués
pending → archived	Montants retirés de `pending_*`	Idem
posted → *	Refusé	Une correction = `POST /reversal`

Contention. Un wallet très actif ne se bloque qu'avec lui-même : le verrou de ligne dure le temps de la transaction (quelques millisecondes). Les comptes partagés par tous les P2P (revenus) sont en mode `async` et ne prennent pas de verrou, ce qui évite que chaque transfert attende le précédent (ADR-05).

5. Sagas des mouvements externes

Chaque mouvement externe est une machine à états persistée dans le service paiements ; chaque transition correspond à un appel ledger unique et rejouable.

Une expiration mène à « inconnu », jamais à un échec automatique



machine à états du cash-out · 8 états

Les fonds ne sont libérés (archived) que sur un échec certain. Un délai dépassé laisse les fonds bloqués jusqu'à la réponse de l'enquête de statut, ce qui évite de rendre à l'utilisateur un argent que l'opérateur a déjà versé (exercice 7).

Les trois sagas

Saga	Appel ledger à la création	Sur succès	Sur échec	`external_id`
Cash-out mobile money	Pending : Débit wallet / Crédit float opérateur, condition `available >= 0`	pending → posted ; frais postés en même temps	pending → archived	`cashout:{order_id}`
Top-up mobile money	Pending : Débit créance opérateur / Crédit wallet, condition `pending <= plafond` + contrôle du cumul mensuel	pending → posted	pending → archived	`topup:{order_id}`

Saga	Appel ledger à la création	Sur succès	Sur échec	`external_id`
PI-SPI sortant	Pending : Débit wallet / Crédit compensation PI-SPI	pending → posted à l'acceptation	pending → archived sur rejet	`pispi-out:{order_id}`
PI-SPI entrant (pas une saga)	—	Posted directement au webhook : Débit compensation / Crédit wallet, ou compte d'attente si alias inconnu	—	`pispi-in:{end_to_end_id}`

Politique d'enquête de statut

- Première enquête 30 s après l'expiration, puis 1, 2, 5 et 10 min (backoff), soit 5 enquêtes.
- Un callback tardif est traité normalement s'il arrive pendant l'état `inconnu`.
- Après 5 enquêtes sans réponse : état `revue manuelle`, alerte ops ; la décision passe par un ajustement à double validation.
- Table `payment_orders(id, type, state, wallet_id, amount, fee, provider, provider_ref, ledger_txn_id, attempts, next_check_at, version)` ; les workers prennent les ordres avec `FOR UPDATE SKIP LOCKED` sur `next_check_at <= now()`.

6. Événements et intégration

Chaque service publie ses faits via une outbox transactionnelle, et chaque consommateur dédoublonne sur l'identifiant d'événement : la livraison est « au moins une fois », l'effet est « exactement une fois ».

6.1 Outbox et publication

- Table `outbox(id, aggregate_type, aggregate_id, event_type, payload jsonb, created_at, published_at)` dans chaque base, écrite dans la même transaction que le changement d'état.

- Un relais par service lit `WHERE published_at IS NULL ORDER BY id FOR UPDATE SKIP LOCKED LIMIT 100`, publie vers RabbitMQ avec confirmations, puis marque `published_at`.
- Chaque consommateur garde une table `processed_events(event_id PRIMARY KEY)` et traite l'événement dans la même transaction que son insertion.

6.2 Catalogue d'événements

Événement	Émetteur	Consommateurs	Usage
`ledger.transaction.pending`	ledger	wallet, notifications	Afficher une opération en cours
`ledger.transaction.posted`	ledger	wallet, conformité, notifications, rapprochement	Historique, AML, SMS, matching
`ledger.transaction.archived`	ledger	wallet, notifications	Blocage libéré
`wallet.created`, `wallet.status_changed`	wallet	paiements, conformité	Alias PI-SPI, gels
`kyc.tier_changed`	wallet (conformité)	wallet	Nouveaux plafonds
`payment_order.status_changed`	paiements	wallet, notifications	Suivi top-up / cash-out
`compliance.alert_raised`	wallet (conformité)	back-office	File d'alertes AML
`recon.break_detected`	rapprochement	back-office	Écart à traiter
`coverage.breach`	rapprochement	wallet, paiements	Gel des émissions et retraits

Enveloppe commune : `{event_id, event_type, occurred_at, trace_id, aggregate_id, version, payload}` ; montants en entiers XOF.

6.3 Idempotence sur trois couches

Couche	Clé	Comportement
Passerelle / API wallet	En-tête `Idempotency-Key` (UUID client)	Table `idempotency_keys(key, request_hash, response, expires_at)` : même clé + même corps → réponse rejouée ; corps différent → 409
Ledger	`external_id` déterministe, ex. `p2p:{transfer_id}`, `cashout:{order_id}:hold`	`INSERT … ON CONFLICT DO NOTHING` puis comparaison de `request_hash`
Callbacks externes	`(provider, provider_event_id)`	Inbox unique ; un doublon est acquitté sans traitement

6.4 Inbox des callbacks

1. Le webhook vérifie la signature et l'horodatage, stocke le corps brut dans `callback_inbox` (contrainte unique sur `provider, provider_event_id`), répond 200 immédiatement.
2. Un worker traite l'inbox dans l'ordre d'arrivée et applique la transition à l'ordre de paiement **seulement si elle avance la machine à états** ; un callback « succès » qui arrive après « expiré » est accepté, un « en cours » arrivant après « succès » est ignoré.
3. Toute transition crée l'appel ledger correspondant avec un `external_id` déterministe : rejouer l'inbox ne peut pas doubler une écriture.

6.5 Appels synchrones

Appelant → appelé	Exemple	Délai	Rejeu
wallet → ledger	Poster un P2P	2 s	Oui, même `external_id`
paiements → ledger	Poser / poster / archiver un blocage	2 s	Oui
paiements → simulateur opérateur	Demande de cash-out	10 s	Non : passage en `unknown` puis enquête de statut

Appelant → appelé	Exemple	Délai	Rejeu
rapprochement → ledger	Soldes de catégories, écritures par compte	30 s	Oui

7. Simulateurs et rapprochement

Les simulateurs sont de vrais services avec leur propre état et une API de contrôle des pannes ; le service de rapprochement les compare au ledger par des jobs planifiés.

7.1 API des simulateurs

Simulateur	API métier (vue par la plateforme)	API de contrôle (vue par les tests)
`sim-pispi`	`POST /transfers` (sortant), `GET /transfers/{id}`, webhook entrant vers la plateforme, `GET /reports/settlement?date=`	`POST /control/inbound` (simuler un virement reçu), `POST /control/faults`
`sim-mno` (deux instances : `mno-wave`, `mno-orange`)	`POST /collections` (top-up), `POST /disbursements` (cash-out), `GET /transactions/{id}`, `GET /float`, callback, `GET /settlement-files?date=`	`POST /control/faults`, `POST /control/float`
`sim-bank` (deux instances : banque A, banque B)	`GET /statements?account=&date=` (CSV dans MinIO)	`POST /control/faults`, `POST /control/credit`

Format d'une panne

```
{
  "operation": "disbursements",
  "mode": "timeout_then_success",
  "probability": 1.0,
  "delay_ms": 120000,
  "count": 1
}
```

Modes : `reject`, `latency`, `timeout_then_success`, `timeout_then_failure`, `duplicate_callback`, `out_of_order_callback`, `float_exhausted`, `missing_statement_line`, `wrong_amount`. Chaque mode est couvert par au moins un test d'intégration (exercices 7 à 10 du CDC).

7.2 Jobs de rapprochement

Job	Déclencheur	Entrées	Sortie
`recon-bank`	Chaque heure + fin de journée	Relevés CSV des deux banques, écritures du compte de cantonnement	Lignes rapprochées, écarts → `recon.break_detected`
`recon-pispi`	Fin de cycle de règlement	Rapport de règlement PI-SPI, solde du compte de compensation	Écarts
`recon-mno`	Quotidien par opérateur	Fichier de règlement, solde de float, créance de règlement	Écarts, alerte float bas
`coverage`	Fin de journée	Soldes des catégories « monnaie en circulation » et « couverture », part à vue	Rapport ; `coverage.breach` si < 100 % ou < 65 % à vue
`drift-scan`	Nocturne + échantillon toutes les 5 min	`ledger_entries` vs `account_balances`	Alerte et reconstruction des compteurs
`balance-aggregator`	Toutes les secondes	`balance_deltas` des comptes asynchrones	Compteurs à jour

Le moteur de rapprochement applique les règles du CDC dans l'ordre : identifiant du rail, puis `external_id`, puis approché (montant + téléphone + fenêtre de 10 min) envoyé en revue humaine. Chaque résultat est stocké dans `recon_matches` / `recon_breaks` avec son statut de traitement.

8. Déploiement, sécurité et observabilité

Tout tourne en local avec Docker Compose ; le découpage permet de passer plus tard sur Kubernetes sans changer le code.

8.1 Services Docker Compose

Conteneur	Image / rôle	Port interne
`gateway`	Traefik : TLS, routage, rate limiting, vérification JWT	443
`keycloak`	OIDC : utilisateurs de test, rôles back-office	8080
`wallet`	Service wallet + module conformité	8001
`ledger`	Service ledger (non exposé par la passerelle)	8002
`payments`	Service paiements + workers saga, inbox, enquêtes de statut	8003
`recon`	Service rapprochement + jobs planifiés	8004
`backoffice`	Web (React ou équivalent)	3000
`postgres`	PostgreSQL 16 : bases `ledger`, `wallet`, `payments`, `recon` avec un rôle par service	5432
`rabbitmq`	Broker, exchanges topic par domaine	5672
`minio`	Relevés, fichiers de règlement, rapports	9000
`sim-pispi`, `mno-wave`, `mno-orange`, `bank-a`, `bank-b`	Simulateurs	9101–9105
`otel-collector`, `prometheus`, `grafana`, `loki`, `tempo`	Observabilité	—

Environnements : `local` (Compose), `ci` (Testcontainers : Postgres, RabbitMQ et simulateurs réels dans les tests d'intégration), `load` (Compose + k6).

8.2 Sécurité

- Seule la passerelle est exposée. Le ledger n'accepte que les appels des services wallet, paiements et rapprochement, authentifiés par jeton de service (client credentials Keycloak) ; mTLS en option.
- Chaque service a son rôle PostgreSQL limité à sa base ; le rôle du ledger n'a ni `UPDATE` ni `DELETE` sur les écritures.
- Les callbacks des simulateurs sont signés en HMAC-SHA256 avec horodatage ; rejet au-delà de 5 min ou en cas de nonce déjà vu.
- Secrets dans un fichier `.env` hors dépôt en local ; Vault ou un gestionnaire de secrets cloud au-delà.
- Ajustements manuels : endpoint back-office qui crée une demande ; un second utilisateur avec le rôle `approver` la valide ; seul ce second appel poste au ledger.

8.3 Observabilité

- Un `trace_id` OpenTelemetry suit chaque requête de la passerelle au ledger ; il est copié dans les métadonnées de la transaction ledger et dans l'enveloppe des événements.
- Métriques minimales : postings par seconde et par type, rejets par motif (`insufficient_funds`, `cap_exceeded`, `unbalanced`, `conflict`), latence p95 du posting, attente de verrou, ordres par état, âge des ordres `unknown`, latence et taux d'échec par simulateur, file outbox non publiée, nombre de dérives, couverture du cantonnement.
- Alertes Grafana : couverture < 100 %, dérive > 0, outbox en retard > 1 min, ordre `unknown` > 30 min, float < seuil.

9. Décisions d'architecture (ADR)

Chaque décision est réversible sauf les trois premières, qui fondent l'exactitude du système.

#	Décision	Alternatives écartées	Raison	Conséquence
ADR-01	Ledger en service et base dédiés	Ledger comme module partagé	Isole l'état monétaire ; un seul code écrit les soldes	Un saut réseau sur chaque mouvement
ADR-02	Écritures immuables, statut porté par la transaction	Mise à jour des écritures	Piste d'audit native, rejouabilité	Corrections par transactions inverses
ADR-03	Compteurs de solde mis à jour dans la transaction de posting	Calcul à la lecture ; cache mis à jour à part	Contrôle de solde en O(1), aucune dérive possible par écriture partielle	Contention sur comptes partagés → ADR-05
ADR-04	Verrous `FOR UPDATE` triés par `account_id`	Verrouillage optimiste seul ; `SERIALIZABLE`	Pas d'interblocage, pas de tempête de rejeux sur un wallet très actif	Le `lock_version` reste exposé pour les clients
ADR-05	Mode `async` pour les comptes chauds sans condition (revenus, compensation, cantonnement)	Tout synchrone ; sous-comptes répartis	Supprime le goulot sur le compte revenus traversé par chaque P2P	Leurs soldes ont quelques secondes de retard ; interdiction de poser une condition dessus
ADR-06	Outbox + RabbitMQ, au moins une fois	Kafka ; double écriture base + broker	Simple en local, pas de perte d'événement	Consommateurs idempotents obligatoires
ADR-07	Sagas orchestrées par le service paiements	Chorégraphie par événements	États explicites, enquêtes de statut et reprises lisibles	Un service porte la logique de chaque flux externe

#	Décision	Alternatives écartées	Raison	Conséquence
ADR-08	Un état `unknown` sur expiration, jamais d'échec automatique	Échec après délai	Évite de libérer des fonds déjà versés par l'opérateur	Fonds bloqués plus longtemps en cas de panne opérateur
ADR-09	Conformité (KYC, plafonds, règles AML) en module du service wallet	Service séparé	Moins de pièces mobiles pour le MVP	Extraction possible par les événements existants
ADR-10	Plafonds et grilles de frais en configuration datée (`valid_from`)	Constantes dans le code	Les plafonds BCEAO changent (2 nov. 2026)	Table de configuration versionnée et auditée
ADR-11	Java 25 + Spring Boot 4.x en monorepo Gradle, un module par couche DDD	Go ; Maven ; un seul module par service	Écosystème mature (Spring Data JDBC, AMQP, Security, Modulith), threads virtuels, frontières DDD vérifiées à la compilation	Quatre modules par contexte ; règles de dépendance testées par ArchUnit
ADR-12	Simulateurs avec API de contrôle des pannes	Mocks dans les tests	Les mêmes pannes servent aux tests, à la démo et à la charge	Code supplémentaire à maintenir

#	Décision	Alternatives écartées	Raison	Conséquence
ADR-13	Persistance du ledger en SQL explicite (JdbcClient ou jOOQ), sans JPA	Spring Data JPA / Hibernate	FOR UPDATE trié, ON CONFLICT et mises à jour de compteurs doivent être visibles et maîtrisés ; pas de flush ni de cache implicite	Plus de SQL à écrire ; JPA reste possible dans wallet et payments
ADR-14	Le domaine est en Java pur : aucune annotation Spring, JPA ou Jackson	Entités annotées utilisées partout	Invariants testables sans conteneur, en millisecondes ; le modèle ne suit pas le schéma	Mapping explicite domaine ↔ persistance dans l'infrastructure

10. Plan d'implémentation

L'ordre de construction suit les étapes du CDC : chaque étape livre des composants et se ferme quand ses exercices passent en CI.

Étape	Composants livrés	Exercices du CDC	Jalon
0 — Fondations	Schéma ledger + migrations, moteur de posting, API ledger, test de propriété, Compose minimal (Postgres)	1, 2, 3	Invariant prouvé
1 — Wallets et P2P	Service wallet (inscription, KYC simulé, niveaux, plafonds, frais), idempotence passerelle, verrous de solde	4, 5, 6	Zéro découvert sous concurrence

Étape	Composants livrés	Exercices du CDC	Jalon
2 — Rails simulés	Simulateurs PI-SPI / opérateurs / banques, service paiements (sagas, inbox, enquêtes), outbox + RabbitMQ, jobs de rapprochement	7, 8, 9, 10	30 jours simulés sans écart
3 — Quasi-production	Back-office (maker-checker, gels, alertes), couverture et rapports, observabilité, comptes asynchrones, tests de charge	11, 12, 13, 14	Charge 10× tenue

Arborescence de départ (monorepo)

```
senwallet/                          # monorepo Gradle (Kotlin DSL)
├── settings.gradle.kts
├── gradle/libs.versions.toml       # versions centralisées
├── build-logic/                    # plugins de convention (java, spring-
service)
├── platform/
│   ├── shared-kernel/              # Money XOF, identifiants, DomainEvent —
Java pur
│   ├── event-contracts/            # événements publiés (records + JSON
Schema)
│   └── test-support/               # Testcontainers, fixtures
├── contexts/
│   ├── ledger/          {domain, application, infrastructure, boot}
│   ├── wallet/          {domain, application, infrastructure, boot}
│   ├── payments/        {domain, application, infrastructure, boot}
│   └── reconciliation/  {domain, application, infrastructure, boot}
├── simulators/          {pispi, mno, bank}   # Spring Boot simples, hors DDD
├── backoffice/
├── contracts/openapi/              # contrats HTTP entre contextes
├── tests/               {property, integration, load}
└── deploy/docker-compose.yml
```

Premier pas concret : créer `build-logic`, `shared-kernel` et `ledger-domain`, puis écrire le test de propriété de l'exercice 1 (jqwik) sur l'agrégat `LedgerTransaction`, avant toute API ni base.

11. Structure du code : monorepo, DDD et Spring Boot

Chaque bounded context est découpé en quatre modules Gradle (domain, application, infrastructure, boot) : les règles de dépendance de l'architecture hexagonale sont ainsi vérifiées à la compilation, puis par ArchUnit.

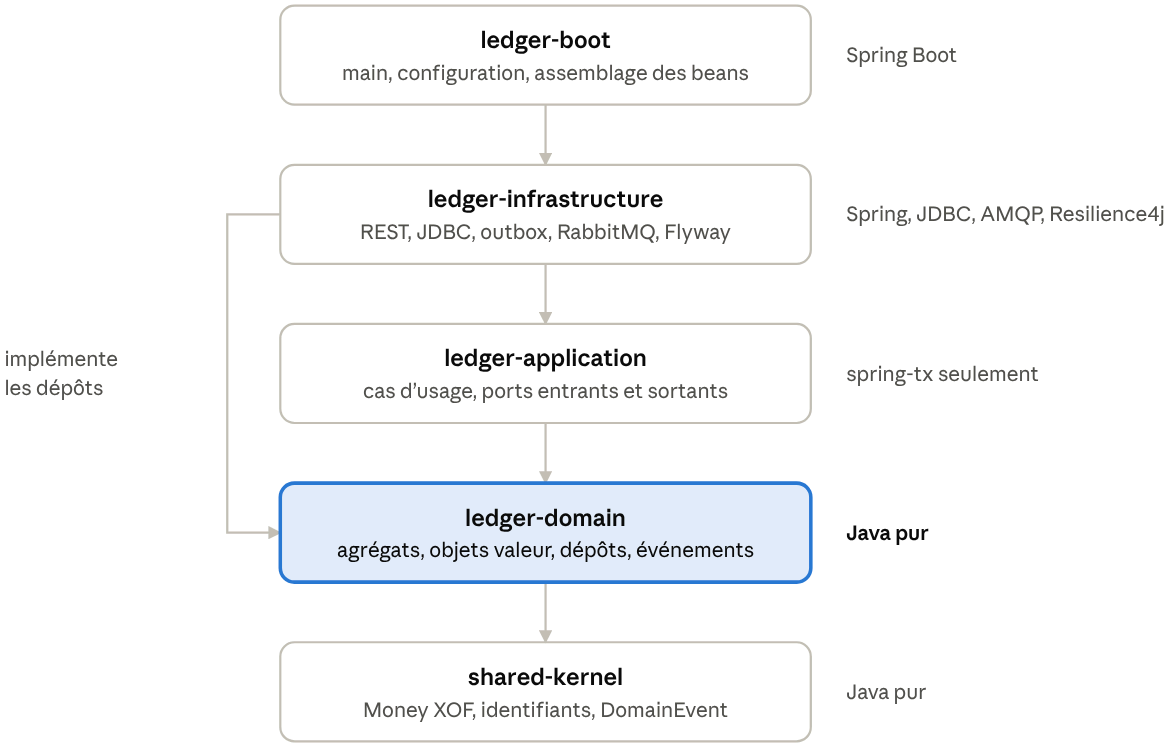
11.1 Bounded contexts

Contexte	Sous-domaine	Agrégats	Relation avec les autres
Ledger	Cœur	`LedgerAccount`, `LedgerTransaction`, `AccountCategory`	Fournisseur : API REST ouverte (Open Host Service) + langage publié (`event-contracts`)
Wallet	Cœur	`Customer` (KYC, niveau), `Wallet`, `Transfer`, `FeeSchedule`, `LimitPolicy`, `ComplianceAlert`	Client du ledger ; publie `wallet.*`
Payments	Support	`PaymentOrder` (machine à états), `CallbackInboxEntry`	Client du ledger ; couche anticorruption par rail (PI-SPI, chaque opérateur)
Reconciliation	Support	`ReconciliationRun`, `Match`, `Break`, `CoverageReport`	Conformiste à l'API du ledger ; couche anticorruption vers les formats de relevés
Simulateurs	Hors domaine	—	Imitent les systèmes externes ; aucun code partagé avec les contextes

Aucun contexte ne dépend des modules d'un autre : ils échangent uniquement par HTTP (clients générés depuis `contracts/openapi`) et par les événements de `event-contracts` . Le scan de dérive et l'agrégateur des comptes asynchrones (section 7.2) tournent dans le service ledger, car ils lisent sa base ; le rapprochement n'en consomme que les résultats.

11.2 Modules et règles de dépendance

Les dépendances pointent vers le domaine, qui ne connaît pas Spring



dépendances entre modules d'un contexte (ledger) · identique pour les quatre contextes

L'infrastructure implémente les ports définis plus bas (dépôts du domaine, ports sortants de l'application) ; le domaine ne voit jamais Spring, JDBC ni Jackson (ADR-14).

```
// contexts/ledger/ledger-domain/build.gradle.kts
plugins { id("senwallet.java-library") }          // aucun plugin Spring
dependencies { implementation(project(":platform:shared-kernel")) }

// contexts/ledger/ledger-application/build.gradle.kts
plugins { id("senwallet.java-library") }
dependencies {
    api(project(":contexts:ledger:ledger-domain"))
    implementation("org.springframework:spring-tx")
}

// contexts/ledger/ledger-boot/build.gradle.kts
plugins { id("senwallet.spring-service") }         // Spring Boot, image OCI
dependencies { implementation(project(":contexts:ledger:ledger-
infrastructure")) }
```

11.3 Packages du contexte ledger

```
contexts/ledger/
├── ledger-domain/        sn.senwallet.ledger.domain
|   ├── model/account/        LedgerAccount, AccountId, NormalBalance,
Balances, BalanceMode
|   ├── model/transaction/    LedgerTransaction, Entry, Direction,
TransactionStatus, ExternalId, Amount
|   ├── model/condition/      BalanceCondition, Operator
|   ├── model/category/       AccountCategory
|   ├── service/              PostingService
|   ├── event/                TransactionPosted, TransactionPending,
TransactionArchived
|   ├── exception/            UnbalancedTransactionException,
BalanceConditionViolatedException
|   └── repository/           LedgerAccountRepository,
LedgerTransactionRepository  (ports)
├── ledger-application/   sn.senwallet.ledger.application
|   ├── port/in/              PostTransactionUseCase,
TransitionTransactionUseCase, BalanceQuery
|   ├── port/out/             OutboxPort, ClockPort
|   ├── command/              PostTransactionCommand, TransitionCommand
|   └── service/              PostTransactionHandler, TransitionHandler,
BalanceQueryService
├── ledger-infrastructure/ sn.senwallet.ledger.infrastructure
|   ├── adapter/in/web/       LedgerTransactionController, DTO, mapping
```

```
ProblemDetail
|   ├── adapter/out/persistence/ JdbcLedgerAccountRepository,
JdbcLedgerTransactionRepository
|   ├── adapter/out/outbox/   JdbcOutboxAdapter, OutboxRelay (@Scheduled),
RabbitEventPublisher
|   ├── job/                  BalanceAggregator, DriftScanner
|   ├── config/               sécurité, Jackson, AMQP
|   └── resources/db/migration/  V1__ledger_schema.sql (Flyway)
└── ledger-boot/          sn.senwallet.ledger
    └── LedgerApplication (@SpringBootApplication), UseCaseConfig (@Bean)
```

Les autres contextes suivent le même gabarit ; dans `payments`, `adapter/out/rails/` contient un adaptateur par rail (`PispiGateway`, `MobileMoneyGateway`) derrière un port `RailGateway`.

11.4 Exemple de code

Domaine — l'agrégat garantit l'équilibre et les transitions, sans framework.

```
package sn.senwallet.ledger.domain.model.transaction;

public final class LedgerTransaction {
    private final TransactionId id;
    private final ExternalId externalId;
    private final List<Entry> entries;
    private TransactionStatus status;
    private final List<DomainEvent> events = new ArrayList<>();

    public static LedgerTransaction open(ExternalId externalId, List<Entry>
entries, TransactionStatus status) {
        if (entries.size() < 2) throw new UnbalancedTransactionException("au
moins deux écritures");
        long debits  = entries.stream().filter(Entry::isDebit).mapToLong(e ->
e.amount().value()).sum();
        long credits = entries.stream().filter(Entry::isCredit).mapToLong(e -
> e.amount().value()).sum();
        if (debits != credits) throw new
UnbalancedTransactionException(debits, credits);
        var txn = new LedgerTransaction(TransactionId.newId(), externalId,
List.copyOf(entries), status);
        txn.events.add(status == POSTED ? new TransactionPosted(txn.id) : new
TransactionPending(txn.id));
        return txn;
```

```
    }

    public void post() {
        if (status != PENDING) throw new IllegalTransitionException(status,
POSTED);
        status = POSTED;
        events.add(new TransactionPosted(id));
    }
}

public record Amount(long value) {                  // XOF, exposant 0
    public Amount { if (value <= 0) throw new
IllegalArgumentException("montant > 0 requis"); }
}
```

Application — le cas d'usage orchestre l'algorithme de la section 4 dans une transaction.

```
package sn.senwallet.ledger.application.service;

@Transactional   // spring-tx : seule dépendance Spring de ce module
public class PostTransactionHandler implements PostTransactionUseCase {
    private final LedgerTransactionRepository transactions;
    private final LedgerAccountRepository accounts;
    private final OutboxPort outbox;
    private final PostingService posting = new PostingService();

    @Override
    public PostingResult handle(PostTransactionCommand cmd) {
        var txn = LedgerTransaction.open(cmd.externalId(), cmd.entries(),
cmd.status());
        if (!transactions.insertHeaderIfAbsent(txn, cmd.requestHash())) {
// ON CONFLICT DO NOTHING
            return
PostingResult.replayOrConflict(transactions.byExternalId(cmd.externalId()),
cmd.requestHash());
        }
        var locked = accounts.lockForUpdateSorted(txn.syncAccountIds());
// FOR UPDATE, ordre fixe
        posting.apply(txn, locked);
// seaux + conditions
        transactions.insertEntries(txn);
        accounts.saveBalances(locked.values());
```

```
        outbox.append(txn.pullEvents());
        return PostingResult.created(txn, locked);
    }
}
```

Infrastructure — SQL explicite (ADR-13).

```
package sn.senwallet.ledger.infrastructure.adapter.out.persistence;

@Repository
class JdbcLedgerAccountRepository implements LedgerAccountRepository {
    private final JdbcClient jdbc;

    @Override
    public Map<AccountId, LedgerAccount>
lockForUpdateSorted(SortedSet<AccountId> ids) {
        return jdbc.sql("""
                SELECT a.id, a.normal_balance, b.posted_dr, b.posted_cr,
b.pending_dr, b.pending_cr, b.lock_version
                FROM ledger_accounts a JOIN account_balances b ON
b.account_id = a.id
                WHERE a.id = ANY(:ids) ORDER BY a.id FOR UPDATE OF b
                """)
            .param("ids",
ids.stream().map(AccountId::value).toArray(UUID[]::new))
            .query(AccountRowMapper.INSTANCE).stream()
            .collect(toMap(LedgerAccount::id, identity()));
    }
}
```

11.5 Stack Spring

Besoin	Choix
Langage et runtime	Java 25 (LTS), threads virtuels activés (`spring.threads.virtual.enabled=true`)
Framework	Spring Boot 4.x, Spring Web MVC
Persistance ledger	`JdbcClient` (ou jOOQ) + Flyway
Persistance autres contextes	Spring Data JDBC (ou JPA) + Flyway

Besoin	Choix
Messagerie	Spring AMQP (RabbitMQ)
Sécurité	Spring Security OAuth2 Resource Server (jetons Keycloak)
Appels sortants	`RestClient` / clients déclarés depuis OpenAPI, Resilience4j (délais, rejeux, disjoncteur)
Observabilité	Actuator, Micrometer Tracing + OpenTelemetry
Tests	JUnit 5, jqwik (propriétés), Testcontainers, ArchUnit, WireMock
Build	Gradle Kotlin DSL, version catalog, plugins de convention dans `build-logic`

11.6 Frontières vérifiées par les tests

```
@AnalyzeClasses(packages = "sn.senwallet")
class ArchitectureTest {
    @ArchTest
    static final ArchRule domainIsPure =
noClasses().that().resideInAPackage("..domain..")
        .should().dependOnClassesThat()
        .resideInAnyPackage("org.springframework..", "jakarta.persistence..",
"com.fasterxml..");

    @ArchTest
    static final ArchRule contextsAreIsolated =
slices().matching("sn.senwallet.(*)..")
        .should().notDependOnEachOther()
        .ignoreDependency(alwaysTrue(),
resideInAPackage("sn.senwallet.shared.."));
}
```

Les deux filets se complètent : Gradle empêche d'importer un module interdit, ArchUnit empêche de contourner la règle par une dépendance transitive.